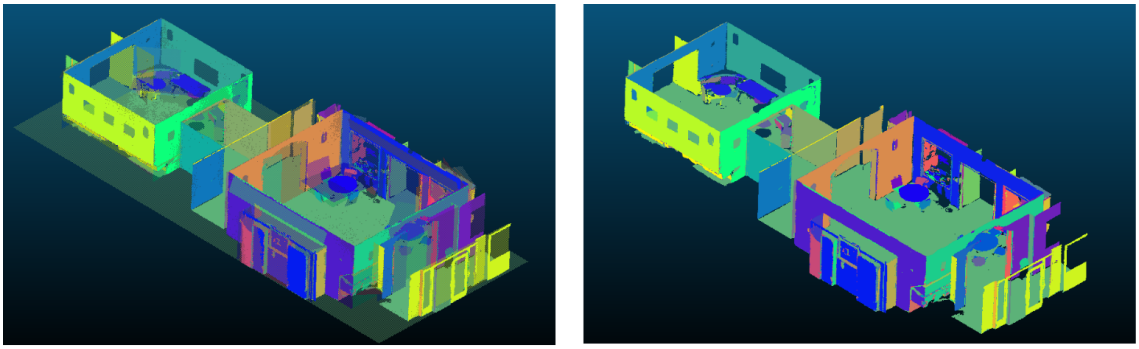
1 Question 1

Fig. 1 shows the best segmentation I obtained using RANSAC plugin in CloudCompare.



(a) (b)

Figure 1: Best segmentation using RANSAC plugin in CloudCompare with (a) and without (b) dip visualization.

To obtain this result, I first examined the `minimum supporting points per primitive` parameter and observed that lowering it can result in too many planes, many of which corresponded to noise or outlier regions rather than meaningful structures. On the contrary, increasing this value resulted in fewer planes, but also increased the possibility of missing smaller surfaces. Therefore, I tested several values in the range of 400-800, and observed that the value of 700 yields finer results compared to, for instance, the default value of 500, which showed more noise in certain regions (e.g., floor, table). Next, I experimented with the `overlooking probability` parameter and observed that increasing it resulted in a segmentation where the planes were detected in the wrong order (i.e., walls of the smaller room were detected before those of the larger room). I therefore kept its value at 0.01. Moreover, I decided to set (and decrease) the `maximum distance to primitive` to 0.08 and the `maximum normal deviation` to 10°, so that the points are closer to their respective planes compared to the default values.

The final segmentation contains 132 planes. The planes are correctly ordered by size, with the largest one corresponding to the floor, followed by the main walls of the larger room, then the smaller room, and subsequently smaller planar structures. Planes that were detected the latest mostly correspond to small surfaces, noise, or outliers, as aforementioned.

Overall, I would say that the segmentation is successful for the most dominant planes (e.g., walls, tables, chairs), but it also produces some smaller noisy and fragmented artifacts, mainly in areas where multiple objects are close to each other and where the point cloud is sparse (e.g., in the corner of the smaller room or near the table in the larger room, ”inside” the wall).

2 Question 2

Fig. 2 shows the result of our Python RANSAC implementation on extracting the first two planes.

We see that the first plane, corresponding to the floor, is correctly detected. However, the second extracted plane is not satisfactory and is not a real planar surface in the scene. Ideally, the extracted planes should match the real surfaces, as in the previous question. Instead, the second plane here is parallel to the floor and made up of points coming from several different surfaces (e.g., walls, tables).

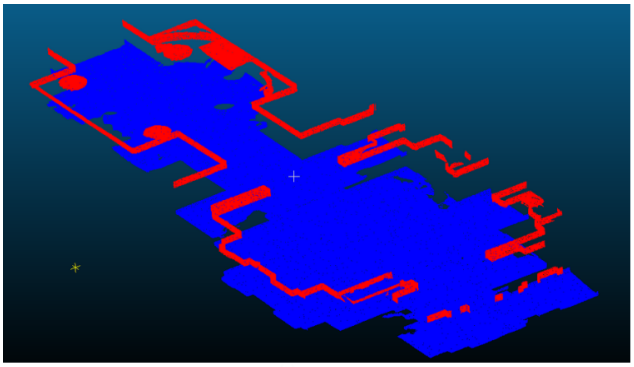


Figure 2: First two extracted planes using Python RANSAC implementation with `nb_draws`=100, `threshold_in`=0.1.

The main problem is that the points belonging to this second plane do not actually belong to the same physical plane and do not share a consistent orientation. This happens because, in our implementation, we only check whether the distance from a point to the estimated plane is smaller than a given threshold. To better understand this behavior, we can increase the `threshold_in` parameter to 0.4 and analyze the result. As shown in Fig. 3, increasing this parameter makes the detected plane appear thicker (i.e. have larger height). This confirms that our inlier criterion does not enforce coplanarity, but instead selects all points that are between two parallel planes placed at a distance of $\pm$`threshold_in` from the estimated plane.

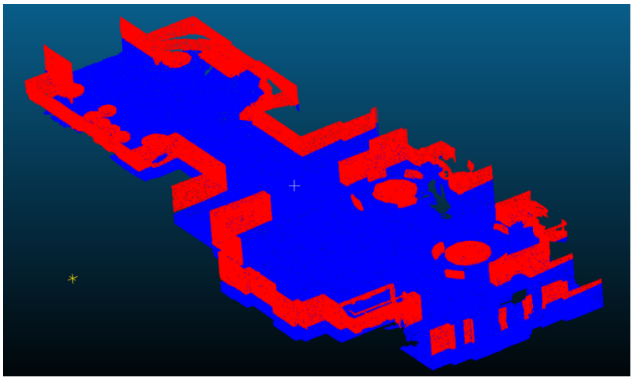


Figure 3: Effect of the `threshold_in` parameter on the extraction of second plane. When increased, the plane gets thicker.

In order to mitigate these problems, we should enforce alignment between the estimated planes and points by introducing the normals information into the RANSAC algorithm (which we do in Question 4).

3 Question 3

For this question, we will use the *Notre_Dame_Des_Champs_1.ply* point cloud. Ideally, we want the first extracted plane to correspond to the street/road, the second one to the building across Notre Dame des Champs, followed by the other buildings, and so on. Since we can reasonably assume what the first two planes should be, we focus on tuning the RANSAC parameters for extracting only these two planes first, and therefore set `nb_planes`=2.

We first run the RANSAC algorithm using the same setup as in the previous question. From the result shown in Fig. 4 (a), we clearly see that extracted planes are not satisfactory. We assume that this is a consequence of higher scene complexity, as the point cloud now contains more planar surfaces than in previous cases. To mitigate this, we choose to increase `nb_draws` to 500 and therefore increase the probability of sampling three points that belong to the same plane. However, presumably due to the presence of noise, increasing this parameter improves the road plane, but still fails at extracting the second one, as shown in Fig. 4 (b). Therefore, we further tune the `threshold_in` parameter and increase it in order to include more points as inliers. We finally set `threshold_in`=0.7, with the corresponding results are shown in Fig. 4 (c).

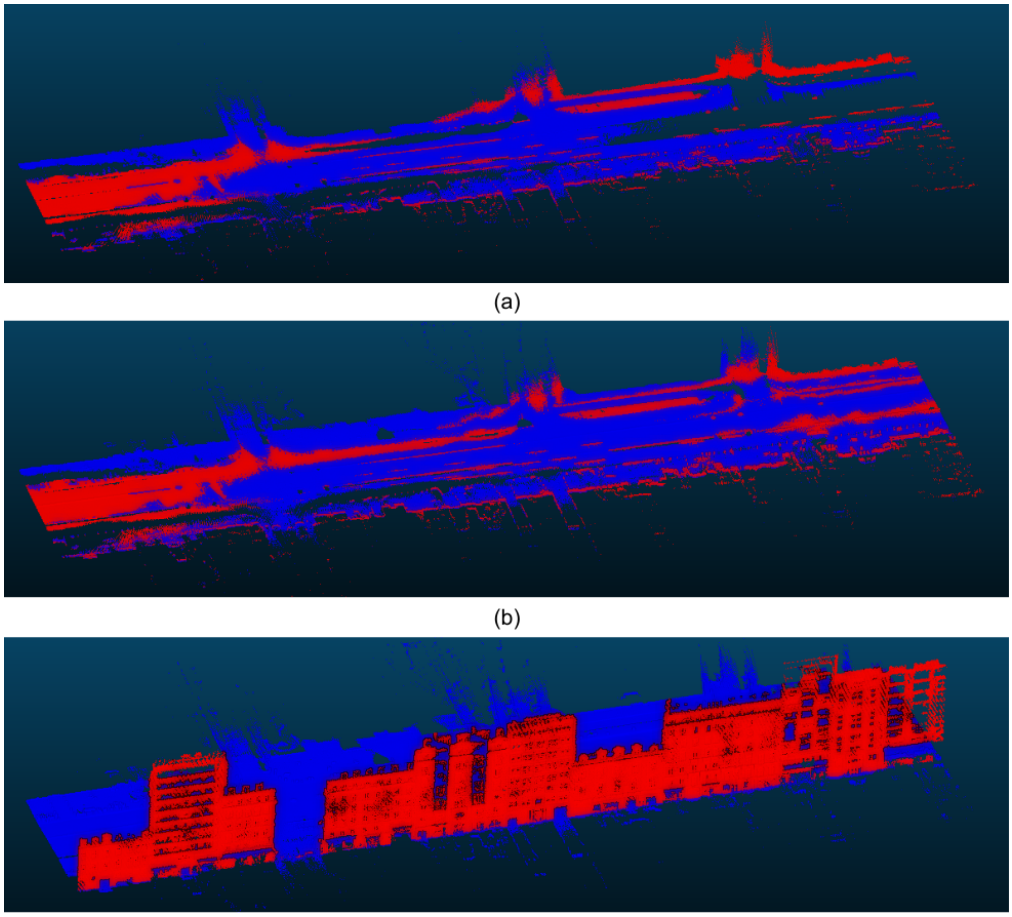
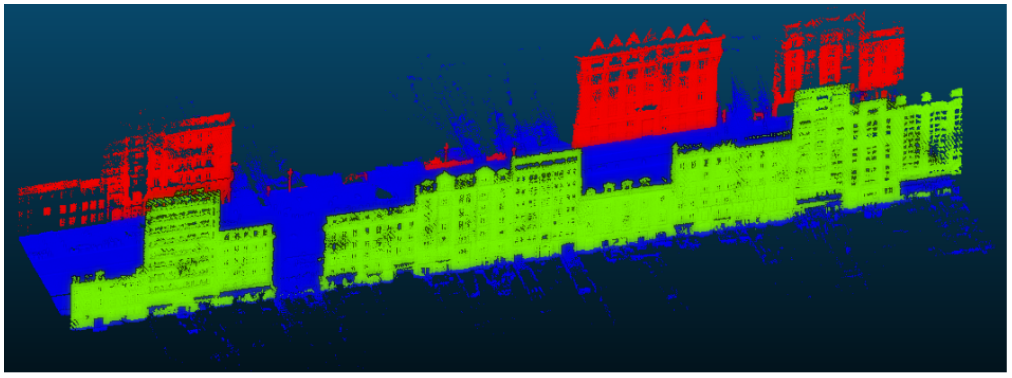


Figure 4: First two extracted planes of *Notre_Dame_Des_Champs_1.ply* using Python RANSAC implementation with different parameter configuration: (a) `nb_draws`=100, `threshold_in`=0.1, (b) `nb_draws`=500, `threshold_in`=0.1 and (c) `nb_draws`=500, `threshold_in`=0.7.

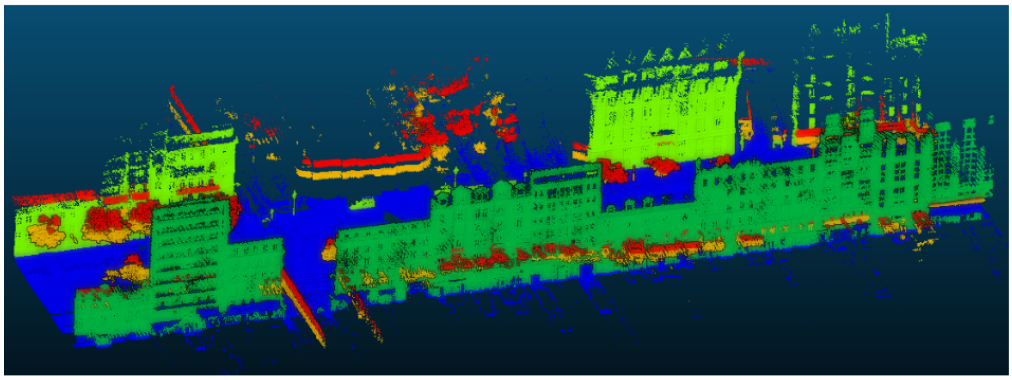
Given that we found suitable parameters for the first two planes, we now use them to extract additional ones. The obtained results are shown in Fig. 5.

We first examine the case when `nb_planes`=3. As hypothesized, the RANSAC algorithm correctly extracts and assigns the first three planes to the road, building across Notre Dame des Champs, and other buildings. However, we note that the third (red) plane does not include all the points from the far left and far right buildings, which we associate with the depth discontinuities on the corresponding facade (i.e., windows).

In the case of `nb_planes`=5, we again clearly see the first three planes, although now, the far right building has significantly fewer points included. If we now just focus on the additional two planes (red and orange), we can observe the same undesirable behavior as in the previous question. Once again, these two planes do not correspond to real surfaces, and instead, they represent approximately horizontal planes that are slightly rotated with respect to the first plane. As a result, they intersect the point cloud and group together points from Notre Dame des Champs, trees and other buildings that lie along the same orientation. Fig. 6 shows this effect more clearly. This behavior is consistent with our previous observations and requires the same adjustment.



(a)



(b)

Figure 5: Extracted planes of *Notre_Dame_Des_Champs_1.ply* using Python RANSAC implementation with (a) `nb_planes=3` and (b) `nb_planes=5`.

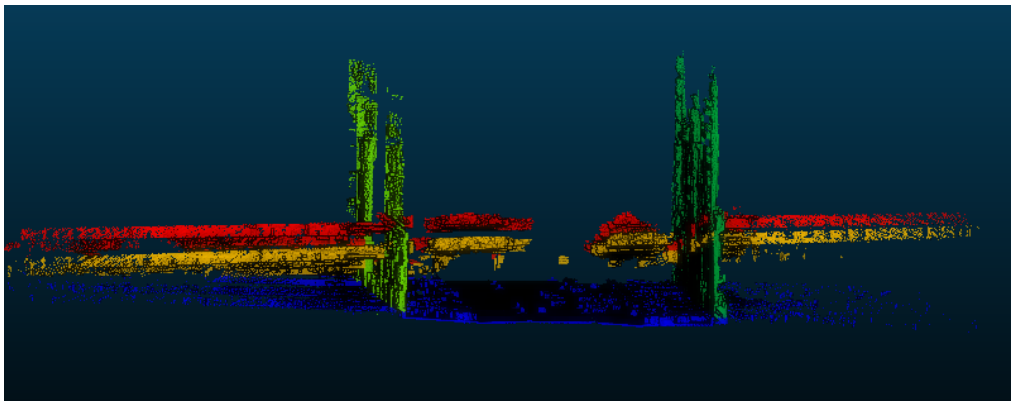


Figure 6: Fourth and fifth extracted planes (orange and red) of *Notre_Dame_Des_Champs_1.ply* that follow the same pattern observed in Question 2.

4 Question 4

To incorporate point normals in the RANSAC algorithm, we first pre-computed normals on the point cloud using a spherical neighborhood search with `radius=0.05`. The radius was set to a relatively small value to limit neighborhood size and reduce memory usage constraints (as higher values resulted in `MemoryError`). I then modified the `in_plane` function to include an additional constraint that checks the angle between the normal at a given point and the normal of the fitted plane. If this angle was lower than the tuned and newly introduced hyperparameter `max_angle`, and the previous distance constraint was also satisfied, the point was considered an inlier. This way, we mitigated the previous issue of having parts of different walls grouped into a single horizontal plane, since all points whose normals are not aligned with the plane's normal are now discarded.

To tune the parameters, we started with the default setup of `nb_planes=5`, `nb_draws=100`, and `threshold_in=0.1`.

The new parameter `max_angle` was set to an initial value of 0.1 (corresponding to 5.7°).

We immediately observed that a lower value of `nb_draws` was insufficient (Fig. 7), and therefore increased it to 500, as in the previous question. We attribute this to the introduction of the additional normal constraints, which reduces the total number of inliers, and therefore, requires more iterations.

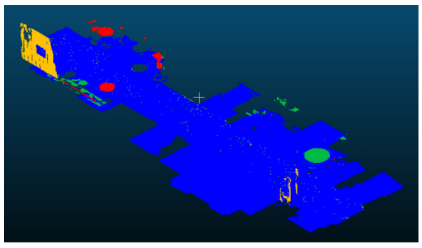


Figure 7: Five extracted planes of *indoor_scan.ply* using Python RANSAC implementation with normals with `nb_draws`=100.

We then tuned the `threshold_in` parameter and observed that higher values (e.g., 0.2 and above) increased the number of detected objects, but at the cost of grouping separate surfaces (e.g., walls) into the same plane (Fig. 8 (a)). This behavior is a consequence of using a larger threshold, which we discussed in Question 2. Therefore, we kept the value at `threshold_in`=0.1.

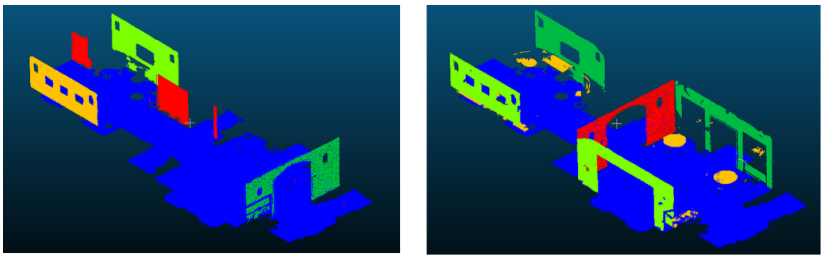


Figure 8: Five extracted planes of *indoor_scan.ply* using Python RANSAC implementation with normals with (a) `threshold_in`=0.1 and (b) `threshold_in`=0.2.

Finally, we examined the `max_angle` parameter and observed that for lower values, the algorithm sometimes failed to include the entire wall surface within a single plane, as shown in Fig. 9 (a). On the other hand, when setting `max_angle`=0.25 (and higher), single walls were usually split into two different planes (Fig. 9 (c)). For both lower and higher values, we additionally observed that separate walls were sometimes grouped into the same plane. Based on these observations, we selected `max_angle`=0.1, as it consistently produced better results compared to the other tested values.

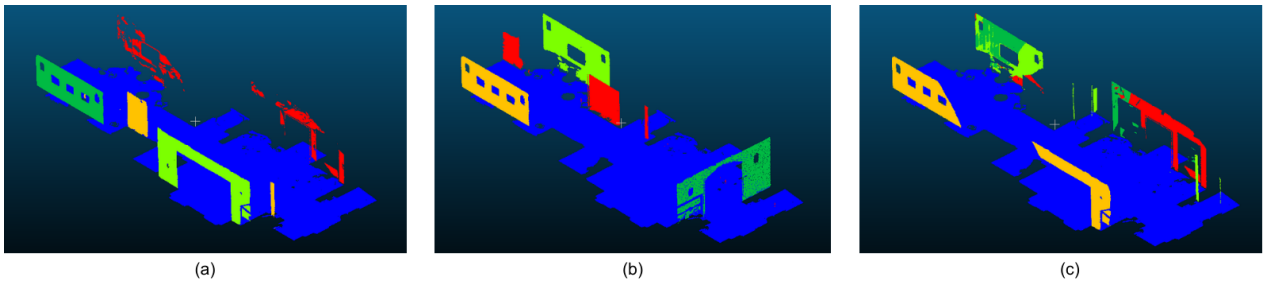


Figure 9: Extracted 5 planes of *indoor_scan.ply* using Python RANSAC implementation with normals with: (a)a `max_angle`=0.05, (b) `max_angle`=0.1, and (c) `max_angle`=0.2.

The final result of our best segmentation is shown in Fig. 10. We clearly observe the plane corresponding to the floor, while the remaining planes correspond to the walls. Another positive aspect is that points assigned to the same plane have consistent normals, which indicates a good segmentation. Moreover, the detected planes,

except for the green one, are not noisy. However, one drawback of the result is that three distinct (but aligned) walls are grouped into a single plane (shown in red), which is not desirable.

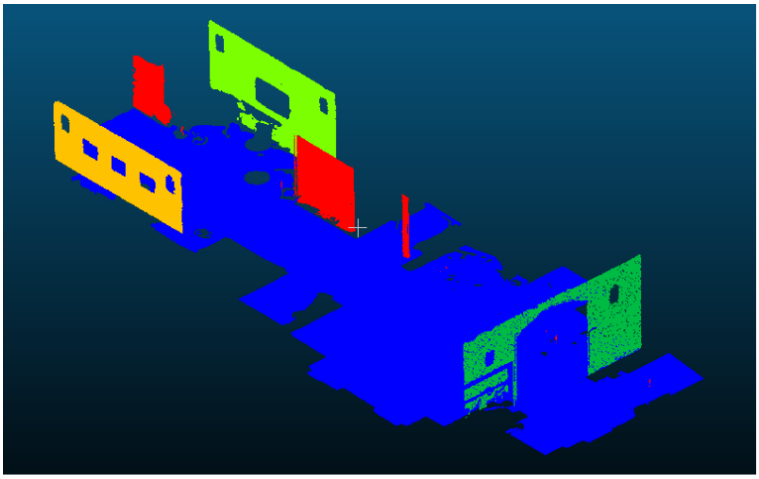


Figure 10: Best segmentation of *indoor_scan.ply* using Python RANSAC implementation with normals.

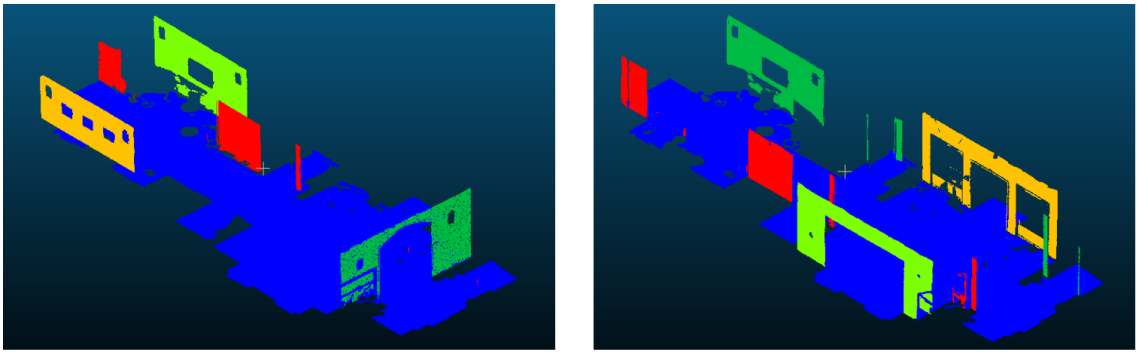
5 Question 5

In our previous runs, we observed that increasing the `nb_draws` parameter significantly increases computation time. However, in certain cases, using a higher value was necessary in order to obtain better results, at the cost of longer computation. Now, since this parameter is directly related to the probability of finding a plane with a large number of inliers, and this probability depends on the initial random sampling of points, we can conclude that improving the sampling strategy itself could allow us to reduce `nb_draws` while still obtaining strong results.

So far, our sampling method simply selected three completely random points in the cloud, which obviously does not take into account the cloud's structure. Instead of doing this, we can observe that planes are local structures, and that points which are close to each other are more likely to belong to the same plane than points that are far apart. To exploit this observation, we can introduce a sampling method based on k-nearest neighbors (k-NN). More precisely, we can select the first point randomly, and then, using a kd-tree, we can query its k nearest neighbors, and then randomly select the remaining two. This approach increases the probability that the three sampled points belong to the same plane, which in turn leads to a more accurate fitted plane. Consequently, we will require fewer RANSAC iterations (i.e., `nb_draws`) to obtain good planes, and thus, good segmentation.

For the final segmentation in this question, we tuned the parameter k in the k-NN based sampling to $k = 1000$, and reduced `nb_draws` from 500 to 100. The resulting segmentation is shown in Fig. 11 (b). All other parameters were set to the same values as in the previous question. The result is similar to the one obtained earlier. We again detect planes corresponding to the floor and the walls, and we observe the same drawback of distinct (but aligned) walls being grouped into the same plane. However, an important difference is that this method produced stable results in every run, unlike the previous one, which depended on random sampling. This is because k-NN based sampling makes the initial three points more meaningful and therefore increases the consistency of the detected planes across runs.

The main advantage of this approach is the significant reduction in computation time. The runtime decreased from ~ 168s to ~ 39s, which corresponds to a reduction by a factor of ~ 4.3. This closely matches the reduction we applied to `nb_draws`, which finally confirms that improving the sampling method allows us to decrease the number of iterations while keeping the same (or even better) segmentation quality.



(a) (b)

Figure 11: Five extracted planes of *indoor_scan.ply* using Python RANSAC implementation with normals (a) without acceleration and (b) with acceleration.